

Repeating Requests

In the previous sections, we successfully bypassed the input validation to use a non-numeric input to reach command injection on the remote server. If we want to repeat the same process with a different command, we would have to intercept the request again, provide a different payload, forward it again, and finally check our browser to get the final result.

As you can imagine, if we would do this for each command, it would take us forever to enumerate a system, as each command would require 5-6 steps to get executed. However, for such repetitive tasks, we can utilize request repeating to make this process significantly easier.

Request repeating allows us to resend any web request that has previously gone through the web proxy. This allows us to make quick changes to any request before we send it, then get the response within our tools without intercepting and modifying each request.

Proxy History

To start, we can view the HTTP requests history in **Burp** at (**Proxy>HTTP History**):

Dashboard	Target	Proxy	Intruder	Repeater	Sequencer	Decoder	Comparer	Logger	Extender	Project options	User options		
Intercept	HTTP history	WebSockets history	Options										
Filter: Hiding CSS, image and general binary content													
#	Host	Method	URL	Params	Edited	Status	Length	MIME type	Extension	Title	Comment	TLS	IP
4	http://46.101.23.188:32505	POST	/ping	✓		200	370	script					46.101.23.188
3	http://46.101.23.188:32505	GET	/favicon.ico			404	394	HTML	ico	Error			46.101.23.188
1	http://46.101.23.188:32505	GET	/			200	1443	HTML		Ping IP			46.101.23.188

In **ZAP** HUD, we can find it in the bottom History pane or ZAP's main UI at the bottom **History** tab as well:

History

WebSockets

Time	Status	Method	URL	Filter	☐ Enable RegEx	0 of 2 items hidden by filters
17:6:37.878	200	POST	http://46.101.23.188:32505/ping			
17:6:39.554	200	GET	http://46.101.23.188:32505/			

Both tools also provide filtering and sorting options for requests history, which may be helpful if we deal with a huge number of requests and want to locate a specific request. **Try to see how filters work on both tools.**

Note: Both tools also maintain WebSockets history, which shows all connections initiated by the web application even after being loaded, like asynchronous updates and data fetching. WebSockets can be useful when performing advanced web penetration testing, and are out of the scope of this module.

If we click on any request in the history in either tool, its details will be shown:

Burp:

Request	Response
<pre>1 POST /ping HTTP/1.1 2 Host: 46.101.23.188:32505 3 Content-Length: 4 4 Cache-Control: max-age=0 5 Upgrade-Insecure-Requests: 1 6 Origin: http://46.101.23.188:32505 7 Content-Type: application/x-www-form-urlencoded 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.114 Safari/537.36 9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9 10 Referer: http://46.101.23.188:32505/ 11 Accept-Encoding: gzip, deflate 12 Accept-Language: en-US,en;q=0.9 13 Connection: close 14 15 ip=1</pre>	<pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Date: 4 Connection: close 5 Content-Length: 251 6 7 PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data. 8 64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.020 ms 9 10 --- 127.0.0.1 ping statistics --- 11 1 packets transmitted, 1 received, 0% packet loss, time 0ms 12 rtt min/avg/max/mdev = 0.020/0.020/0.020/0.000 ms 13</pre>

ZAP:

Request	Response
<pre>POST http://46.101.23.188:32505/ping HTTP/1.1 User-Agent: Mozilla/5.0 20100101 Firefox/90.0 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8 Accept-Language: en-US,en;q=0.5 Content-Type: application/x-www-form-urlencoded Content-Length: 4</pre>	<pre>HTTP/1.1 200 OK X-Powered-By: Express Date: Connection: keep-alive PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data. 64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.021 ms</pre>

```

Origin: https://46.101.23.188:32505
Connection: keep-alive
Referer: https://46.101.23.188:32505/
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Host: 46.101.23.188:32505

ip=1

```

```

--- 127.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.021/0.021/0.021/0.000 ms

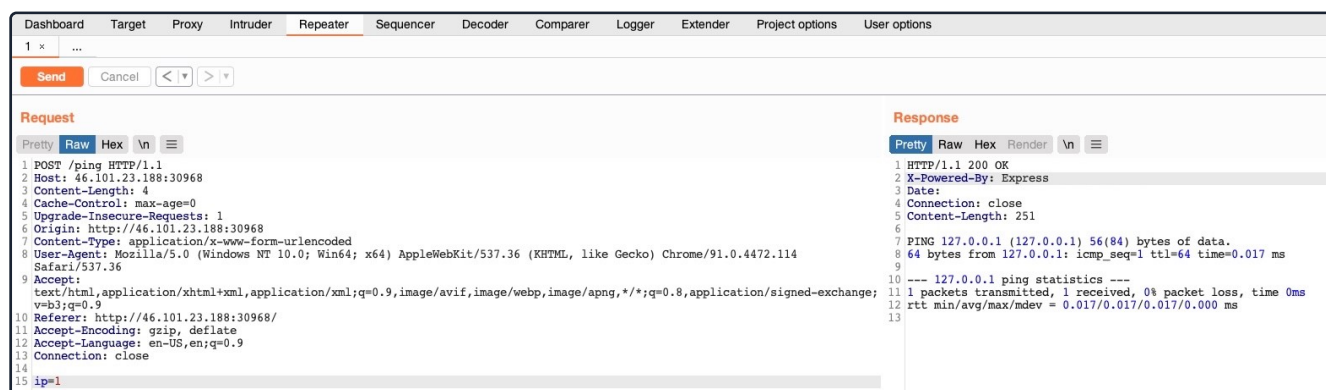
```

Tip: While ZAP only shows the final/modified request that was sent, Burp provides the ability to examine both the original request and the modified request. If a request was edited, the pane header would say **Original Request**, and we can click on it and select **Edited Request** to examine the final request that was sent.

Repeating Requests

Burp

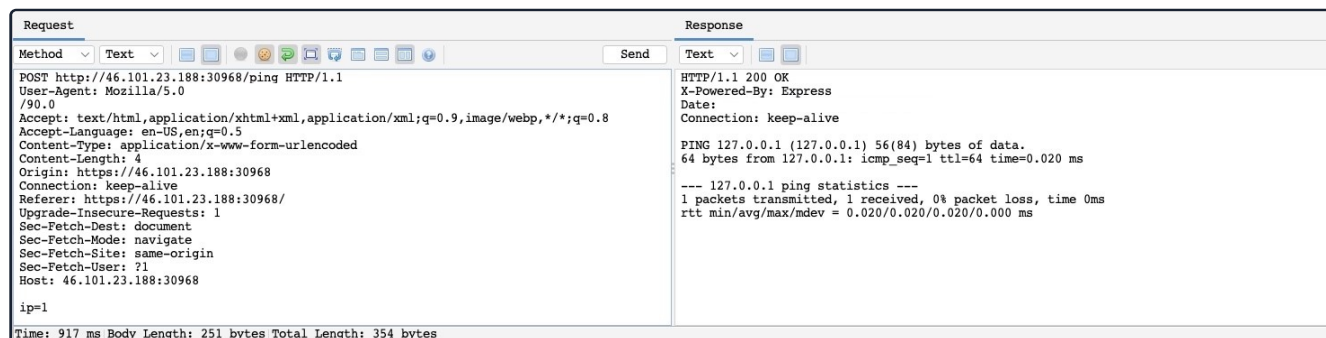
Once we locate the request we want to repeat, we can click **[CTRL+R]** in Burp to send it to the **Repeater** tab, and then we can either navigate to the **Repeater** tab or click **[CTRL+SHIFT+R]** to go to it directly. Once in **Repeater**, we can click on **Send** to send the request:



Tip: We can also right-click on the request and select **Change Request Method** to change the HTTP method between POST/GET without having to rewrite the entire request.

ZAP

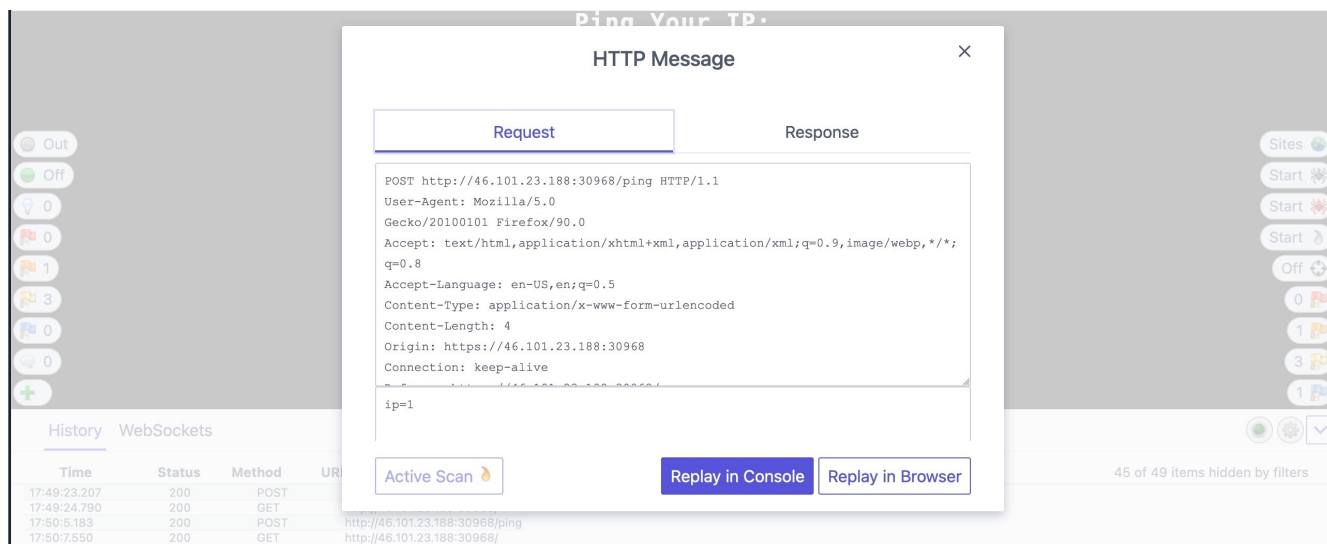
In ZAP, once we locate our request, we can right-click on it and select **Open/Resend with Request Editor**, which would open the request editor window, and allow us to resend the request with the **Send** button to send our request:



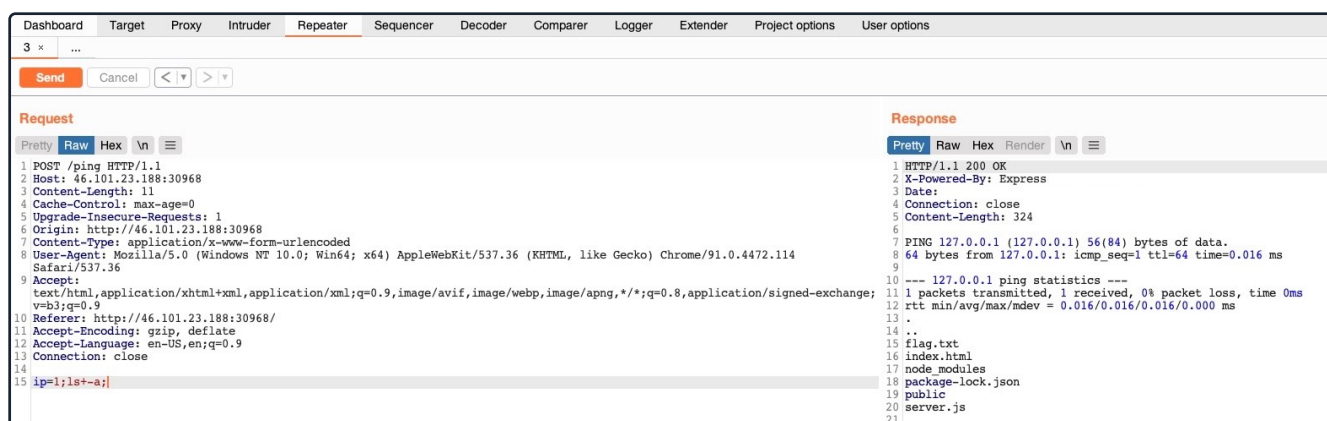
We can also see the **Method** drop-down menu, allowing us to quickly switch the request method to any other HTTP method.

Tip: By default, the Request Editor window in ZAP has the Request/Response in different tabs. You can click on the display buttons to change how they are organized. To match the above look choose the same display options shown in the screenshot.

We can achieve the same result within the pre-configured browser with **ZAP HUD**. We can locate the request in the bottom History pane, and once we click on it, the **Request Editor** window will show, allowing us to resend it. We can select **Replay in Console** to get the response in the same HUD window, or select **Replay in Browser** to see the response rendered in the browser:



So, let us try to modify our request and send it. In all three options ([Burp Repeater](#), [ZAP Request Editor](#), and [ZAP HUD](#)), we see that the requests are modifiable, and we can select the text we want to change and replace it with whatever we want, and then click the [Send](#) button to send it again:



As we can see, we could easily modify the command and instantly get its output by using [Burp Repeater](#). Try doing the same in [ZAP Request Editor](#) and [ZAP HUD](#) to see how they work.

Finally, we can see in our previous POST request that the data is URL-encoded. This is an essential part of sending custom HTTP requests, which we will discuss in the next section.



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

40ms

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions



Cheat Sheet

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)



+ 1

Try using request repeating to be able to quickly test commands. With that, try looking for the other flag.

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution

← Previous

Next →



Cheat Sheet



Go to Questions

Table of Contents

Getting Started

Intro to Web Proxies



Setting Up



Web Proxy

Proxy Setup



Intercepting Web Requests



Intercepting Responses



Automatic Modification



[Repeating Requests](#)

Encoding/Decoding

Proxying Tools

Web Fuzzer

 Burp Intruder

 ZAP Fuzzer

Web Scanner

Burp Scanner

 ZAP Scanner

Extensions

Skills Assessment

 Skills Assessment - Using Web Proxies

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left